

AvgPool3d#

<https://pytorch.org/docs/stable/generated/torch.nn.AvgPool3d.html>

Description:

Applies a 3D average pooling over an input signal composed of several input planes. In the simplest case, the output value of the layer with input size (N, C, D, H, W) , output $(N, C, D_{\text{out}}, H_{\text{out}}, W_{\text{out}})$ and kernel_size (kD, kH, kW) can be precisely described as: If padding is non-zero, then the input is implicitly zero-padded on all three sides for padding number of points. When `ceil_mode=True`, sliding windows are allowed to go off-bounds if they start within the left padding or the input. Sliding windows that would start in the right padded region are ignored.

Function Signature

```
class torch.nn.AvgPool3d(kernel_size, stride=None,
padding=0, ceil_mode=False, count_include_pad=True,
divisor_override=None)[source]#
```

Parameters

Shape:

```
forward(input)[source]#
```

Return type

Returns:

Tensor

Code Examples

```
>>> # pool of square window of size=3, stride=2
>>> m = nn.AvgPool3d(3, stride=2)
>>> # pool of non-square window
>>> m = nn.AvgPool3d((3, 2, 2), stride=(2, 1, 2))
>>> input = torch.randn(20, 16, 50, 44, 31)
>>> output = m(input)
```

Notes & Warnings

NOTE

Note When `ceil_mode=True`, sliding windows are allowed to go off-bounds if they start within the left padding or the input. Sliding windows that would start in the right padded region are ignored.

NOTE

Note pad should be at most half of effective kernel size.

Detailed Documentation

Documentation

Applies a 3D average pooling over an input signal composed of several input planes.

In the simplest case, the output value of the layer with input size (N, C, D, H, W) (N, C, D, H, W), output $(N, C, D_{\text{out}}, H_{\text{out}}, W_{\text{out}})$ ($N, C, D_{\text{out}}, H_{\text{out}}, W_{\text{out}}$) and kernel_size (kD, kH, kW) (kD, kH, kW) can be precisely described as:

If padding is non-zero, then the input is implicitly zero-padded on all three sides for padding number of points.

When `ceil_mode=True`, sliding windows are allowed to go off-bounds if they start within the left padding or the input. Sliding windows that would start in the right padded region are ignored.

pad should be at most half of effective kernel size.

The parameters kernel_size, stride can either be:

a single int – in which case the same value is used for the depth, height and width dimension

a tuple of three ints – in which case, the first int is used for the depth dimension, the second int for the height dimension and the third int for the width dimension

kernel_size (Union[int, tuple[int, int, int]]) – the size of the window

stride (Union[int, tuple[int, int, int]]) – the stride of the window. Default value is kernel_size

padding (Union[int, tuple[int, int, int]]) – implicit zero padding to be added on all three sides

ceil_mode (bool) – when True, will use ceil instead of floor to compute the output shape

count_include_pad (bool) – when True, will include the zero-padding in the averaging calculation

divisor_override (Optional[int]) – if specified, it will be used as divisor, otherwise kernel_size will be used

Input: (N,C,Din,Hin,Win)(N, C, D_{in}, H_{in}, W_{in})(N,C,Din,Hin,Win) or (C,Din,Hin,Win)(C, D_{in}, H_{in}, W_{in})(C,Din,Hin,Win).

Output: (N,C,Dout,Hout,Wout)(N, C, D_{out}, H_{out}, W_{out})(N,C,Dout,Hout,Wout) or (C,Dout,Hout,Wout)(C, D_{out}, H_{out}, W_{out})(C,Dout,Hout,Wout), where

Per the note above, if ceil_mode is True and $(D_{out}-1) \times \text{stride}[0] \geq D_{in} + \text{padding}[0]$ (D_{out}

- $1) \times \text{stride}[0] \geq D_{\text{in}} + \text{padding}[0](D_{\text{out}} - 1) \times \text{stride}[0] \geq D_{\text{in}} + \text{padding}[0]$, we skip the last window as it would start in the padded region, resulting in D_{out} being reduced by one.

The same applies for W_{out} and H_{out} .

Runs the forward pass.